Principaux algorithmes de Machine Learning

Fondements, principes détaillés, notations et applications Python

Support universitaire – Licence avancée et Master

2026

Objectifs et acquis attendus

- Choisir un paradigme en fonction d'une problématique réelle.
- Comprendre l'intuition, les hypothèses et les équations de 29 méthodes.
- Interpréter chaque symbole, paramètre appris et hyperparamètre.
- Mettre en œuvre les approches sous Python avec évaluation rigoureuse.
- Identifier le surapprentissage, les fuites de données et les limites métier.

Plan général — Partie 1

- 1. Fondements : données, apprentissage, généralisation et métriques.
- 2. Régressions : linéaire, pénalisées, polynomiale, arbres et ensembles.
- 3. Classification : logistique, KNN, Bayes, arbres, SVM, boosting.

Plan général — Partie 2

- 4. Non supervisé : partitionnement, ACP, projections et anomalies.
- 5. Renforcement : bandits, exploration, Q-learning, SARSA et DQN.
- 6. Comparaisons méthodologiques et choix des modèles.
- 7. Travaux pratiques et exercices ; annexes bibliographiques.

Science des données : démarche

- Question économique et indicateur à améliorer
- Collecte, qualité, exploration et connaissance métier
- Choix d'une base de comparaison avant l'algorithme
- Expérimentation, évaluation, déploiement et suivi

Trois grands paradigmes

- Supervisé : paires (x,y), prédire une cible observée
- Non supervisé : structure de X sans utiliser de cible
- Renforcement : agent, état, action, récompense, politique

Forme des données

- X : matrice de n lignes et p caractéristiques
- y : vecteur cible de longueur n (si supervisé)
- Observations indépendantes ou structurées temporellement
- Numériques, catégorielles, textuelles ou séquentielles

Paramètre appris et hyperparamètre

- Paramètre appris : coefficient beta, centroïde mu ou poids neuronal
- Hyperparamètre : alpha de Ridge, K de K-means, profondeur d'arbre
- Les hyperparamètres sont choisis via validation, jamais sur le test final

Fonction de coût

- La perte mesure le désaccord entre prédiction et vérité
- L'optimiseur cherche à minimiser l'erreur et éventuellement une pénalité
- Attention au choix de l'unité et de la métrique métier

Descente de gradient

- Calculer le gradient par rapport aux paramètres
- Faire une mise à jour de pas eta
- Réitérer jusqu'à convergence ou nombre maximal d'itérations

Sous-apprentissage

- Modèle trop simple : erreurs élevées sur entraînement et validation
- Envisager variables plus informatives ou modèle plus flexible

Surapprentissage

- Erreur d'entraînement très faible, validation dégradée
- Remèdes : validation correcte, régularisation, moins de complexité

Biais–variance

- Biais élevé : classe de modèles rigide
- Variance élevée : modèle sensible aux perturbations de l'échantillon
- Le bruit irréductible limite les performances

Séparation des données

- Entraînement : ajustement des paramètres
- Validation : réglage des hyperparamètres
- Test : évaluation finale et indépendante une seule fois

Validation croisée

- K plis : répéter les ajustements sur partitions distinctes
- Stratifier si classes déséquilibrées
- GroupKFold si observations liées, TimeSeriesSplit pour chronologie

Fuite de données

- Ne jamais standardiser ou imputer avec le test
- Écarter les prédicteurs indisponibles à la date de décision
- Utiliser Pipeline et ColumnTransformer

Standardisation

- Mettre les variables sur des échelles comparables
- Indispensable pour KNN, SVM, Ridge, PCA ; souvent secondaire pour arbres

Échantillonnage déséquilibré

- Ne pas juger uniquement par accuracy
- Utiliser précision, rappel, F1, PR-AUC et courbes de calibration

Validation temporelle

- Les données futures ne doivent jamais influencer la préparation passée
- Utiliser un découpage chronologique ou une validation glissante

Interprétation et prudence

- Importance prédictive ne prouve pas une cause
- Comparer les groupes, inspecter dérive, documenter les limites

MAE : erreur absolue moyenne

Formule

$$\text{MAE} = \frac{1}{n} \sum_{i} |y_i - \widehat{y}_i|$$

Paramètres

n : nombre d'observations ; y_i : valeur vraie ; y-hat_i : prédiction ; |.| : valeur absolue.

Exemple / interprétation

Exemple : vrais (2,4), prédits (3,6) => MAE=1,5.

MSE : erreur quadratique moyenne

Formule

$$\text{MSE} = \frac{1}{n}\sum_{i}(y_i - \widehat{y}_i)^2$$

Paramètres

n : observations ; y_i : vérité ; y-hat_i : prévision ; carré : fortes erreurs davantage pénalisées.

Exemple / interprétation

Erreurs 1 et 2 => MSE=(1+4)/2=2,5.

RMSE : racine MSE

Formule

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \widehat{y}_i)^2}$$

Paramètres

n : observations ; y_i : vérité ; y-hat_i : estimation ; sqrt : racine carrée pour revenir à l'unité de y.

Exemple / interprétation

Pour MSE=2,5, RMSE≈1,58.

R^2 : pouvoir descriptif

Formule

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Paramètres

y-bar : moyenne des valeurs y ; numérateur : somme des carrés des résidus ; dénominateur : variabilité totale.

Exemple / interprétation

R^2=0 signifie performance égale à la moyenne de référence sur ce jeu.

Précision, rappel, F1

Formule

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}$$

Paramètres

TP : vrais positifs ; FP : faux positifs ; FN : faux négatifs ; P : précision ; R : rappel ; F1 : moyenne harmonique.

Exemple / interprétation

TP=8, FP=2, FN=4 : précision=0,8 ; rappel≈0,667 ; F1≈0,727.

Exactitude et matrice de confusion

Formule

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Paramètres

TP,TN : bonnes prédictions positive/négative ; FP,FN : erreurs ; dénominateur : total.

Exemple / interprétation

Sur 100 dont 95 négatifs, prédire toujours négatif donne 95 % sans détecter un positif.

ROC-AUC et PR-AUC

Formule

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

Paramètres

TPR : rappel ; FPR : taux de faux positifs ; ROC : TPR contre FPR selon le seuil ; PR : précision contre rappel.

Exemple / interprétation

Lorsque positifs rares, la PR-AUC reflète mieux la qualité des alertes.

Formule

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

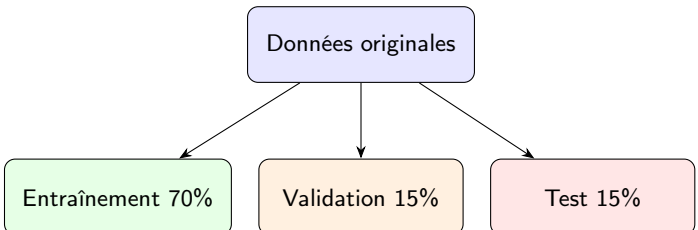
Paramètres

a(i) : distance moyenne aux points du groupe i ; b(i) : meilleure distance moyenne aux autres groupes ; s(i) : coefficient de silhouette.

Exemple / interprétation

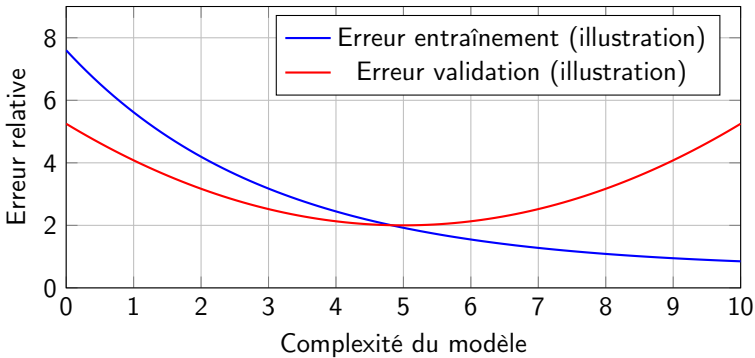
Si a=2 et b=6 : silhouette=4/6≈0,667.

Schéma — protocole sans fuite



Les proportions sont illustratives ; le schéma s'adapte aux données temporelles.

Schéma — compromis biais–variance



Plan — 2. Algorithmes de régression

- Expliquer le principe opérationnel des 29 algorithmes, étape par étape.
- Relier intuition, fonctionnement, équations et paramètres.
- Calcul numérique, schéma et exemple Python.
- Discussion : limites, métriques et applications.

Définition

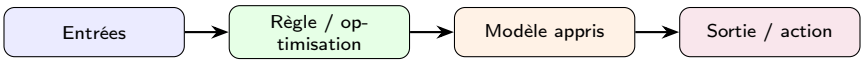
Ajuster une droite ou un hyperplan qui minimise les erreurs quadratiques.

- Problème étudié : Prévision du chiffre d'affaires
- Famille : régression supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

Principe : Régression ordinaire par moindres carrés

Construire la matrice des caractéristiques et ajouter une constante.

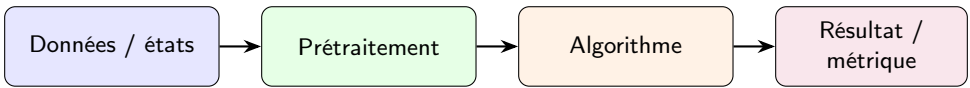
1. Prédire une valeur par une combinaison pondérée des caractéristiques.
2. Chercher les coefficients qui minimisent la somme des carrés des erreurs.
3. Utiliser les coefficients estimés pour prédire de nouvelles observations.



Point de vigilance

Les coefficients décrivent des variations conditionnelles, sans établir de causalité.

1. Régression linéaire — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Ajuster une droite ou un hyperplan qui minimise les erreurs quadratiques.

Modèle / critère principal

$$\widehat{y}_i = \beta_0 + \sum_{j=1}^{p} \beta_j x_{ij}, \quad J = \frac{1}{2n} \sum_{i=1}^{n} (y_i - \widehat{y}_i)^2$$

Lecture pédagogique

Ajuster une droite ou un hyperplan qui minimise les erreurs quadratiques.

Tous les symboles de l'équation

n : nombre de lignes ; p : nombre de caractéristiques ; x_{ij} : variable j du client i ; y_i : vraie valeur ; beta_0 : constante ; beta_j : coefficient estimé ; J : coût moyen quadratique.

Paramètres et hyperparamètres

Hyperparamètre principal : fit_intercept. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

Exemple numérique

Avec beta_0=2, beta_1=3 et x=4 : prediction=14. Si y=15, erreur=1.

Question de compréhension

Quelle modification de fit_intercept pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
LinearRegression()
```

- Avantage : Rapide, explicable
- Limite : Sensibilité aux valeurs atypiques et à la non-linéarité
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

2. Régression Ridge — intuition

Définition

Régulariser les coefficients par une pénalité quadratique.

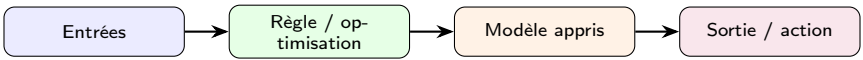
- Problème étudié : Prévision avec variables économiques corrélées
- Famille : régression supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

2. Régression Ridge — principe de fonctionnement

Principe : Régularisation L2

Standardiser les variables pour rendre la pénalité comparable.

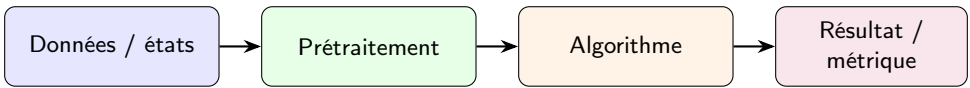
1. Calculer les prédictions du modèle linéaire.
2. Minimiser erreur quadratique et pénalité L2 sur les coefficients.
3. Choisir la force de régularisation par validation croisée.



Point de vigilance

La pénalité rapproche les coefficients de zéro sans les annuler généralement.

2. Régression Ridge — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Régulariser les coefficients par une pénalité quadratique.

2. Régression Ridge — équation

Modèle / critère principal

$$J(\beta) = \frac{1}{n}\|y - X\beta\|_2^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

Lecture pédagogique

Régulariser les coefficients par une pénalité quadratique.

2. Régression Ridge — notations

Tous les symboles de l'équation

X : matrice n x p ; y : vecteur n x 1 ; beta : vecteur p x 1 (interception exclue de la pénalité) ; lambda : force de régularisation ; ||.||_2 : norme euclidienne.

Paramètres et hyperparamètres

Hyperparamètre principal : alpha. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

2. Régression Ridge — exemple calculé

Exemple numérique

Si beta=(2,1) et lambda=0,5, la pénalité vaut 0,5(4+1)=2,5.

Question de compréhension

Quelle modification de alpha pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
Ridge(alpha=1.0)
```

- Avantage : Stable avec colinéarité
- Limite : Ne sélectionne pas automatiquement des variables
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

Définition

Favoriser une solution parcimonieuse avec pénalité absolue.

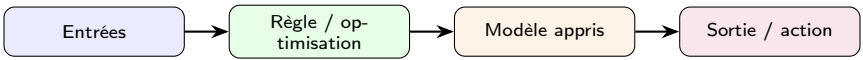
- Problème étudié : Sélection de ratios financiers
- Famille : régression supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

3. Régression Lasso — principe de fonctionnement

Principe : Régularisation L1

Centrer et mettre les caractéristiques à la même échelle.

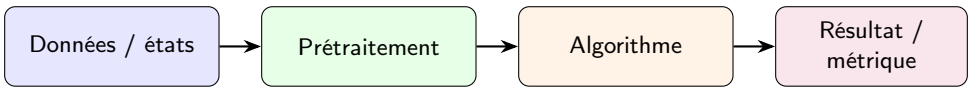
1. Produire des prédictions linéaires.
2. Minimiser erreur quadratique et somme des valeurs absolues des coefficients.
3. Sélectionner la pénalité sur validation et examiner les coefficients nuls.



Point de vigilance

L1 permet une sélection de variables, instable en présence de forts liens entre variables.

3. Régression Lasso — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Favoriser une solution parcimonieuse avec pénalité absolue.

3. Régression Lasso — équation

Modèle / critère principal

$$J(\boldsymbol{\beta}) = \frac{1}{2n}\|\mathrm{y} - X\boldsymbol{\beta}\|_2^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

Lecture pédagogique

Favoriser une solution parcimonieuse avec pénalité absolue.

3. Régression Lasso — notations

Tous les symboles de l'équation

n : taille de l'échantillon ; X : matrice explicative ; y : cible ; beta_j : coefficients ; lambda : pénalité L1 ; |.| : valeur absolue.

Paramètres et hyperparamètres

Hyperparamètre principal : alpha. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

Exemple numérique

Si beta=(2,-1) et lambda=0,5, la pénalité vaut 1,5.

Question de compréhension

Quelle modification de alpha pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
Lasso(alpha=0.1, max_iter=10000)
```

- Avantage : Sélection intégrée
- Limite : Instable en présence de variables très corrélées
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

4. Régression polynomiale — intuition

Définition

Ajouter des puissances et interactions pour modéliser une courbe.

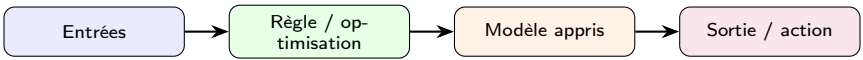
- Problème étudié : Évolution non linéaire des prix
- Famille : régression supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

4. Régression polynomiale — principe de fonctionnement

Principe : Transformation des caractéristiques

Créer des puissances et interactions des caractéristiques.

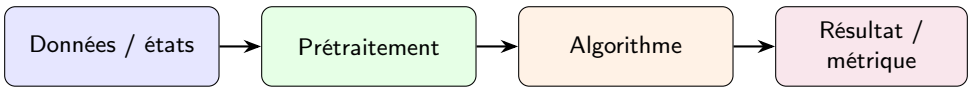
1. Appliquer un modèle linéaire à cet espace enrichi.
2. Estimer ses coefficients sur le jeu d'entraînement.
3. Choisir le degré sur validation et contrôler le surapprentissage.



Point de vigilance

Polynomiale dans les variables ne veut pas dire non linéaire dans les coefficients.

4. Régression polynomiale — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Ajouter des puissances et interactions pour modéliser une courbe.

4. Régression polynomiale — équation

Modèle / critère principal

$$\widehat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d$$

Lecture pédagogique

Ajouter des puissances et interactions pour modéliser une courbe.

4. Régression polynomiale — notations

Tous les symboles de l'équation

x : variable d'entrée ; d : degré maximal ; beta_j : paramètres appris ; y-hat : valeur estimée.

Paramètres et hyperparamètres

Hyperparamètre principal : degree. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

4. Régression polynomiale — exemple calculé

Exemple numérique

Avec beta=(1,2,0,5) et x=2 : prediction=1+4+2=7.

Question de compréhension

Quelle modification de degree pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
make_pipeline(PolynomialFeatures(2), LinearRegression())
```

- Avantage : Modélise courbes simples
- Limite : Explose en dimension et peut surapprendre
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

5. Arbre de régression — intuition

Définition

Découper l'espace en feuilles et prédire leur moyenne.

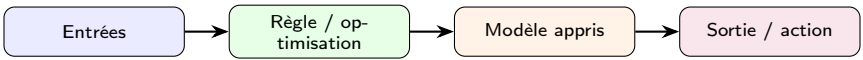
- Problème étudié : Estimation des coûts logistiques
- Famille : régression supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

5. Arbre de régression — principe de fonctionnement

Principe : Découpage récursif

Examiner plusieurs variables et seuils de coupure.

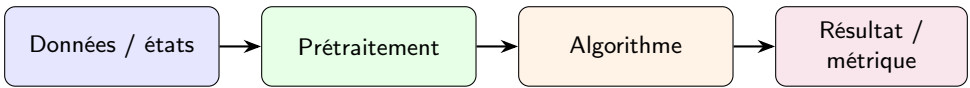
1. Retenir la séparation qui réduit le plus la dispersion des cibles.
2. Répéter récursivement dans les sous-ensembles.
3. Prédire la moyenne des cibles dans la feuille atteinte.



Point de vigilance

Profondeur et taille minimale des feuilles contrôlent la complexité.

5. Arbre de régression — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Découper l'espace en feuilles et prédire leur moyenne.

5. Arbre de régression — équation

Modèle / critère principal

$$\widehat{y}(x) = \frac{1}{|L(x)|} \sum_{i \in L(x)} y_i$$

Lecture pédagogique

Découper l'espace en feuilles et prédire leur moyenne.

5. Arbre de régression — notations

Tous les symboles de l'équation

L(x) : indices de la feuille contenant x ; |L(x)| : effectif ; y_i : valeur des observations dans la feuille.

Paramètres et hyperparamètres

Hyperparamètre principal : max_depth. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

5. Arbre de régression — exemple calculé

Exemple numérique

Dans une feuille contenant 8, 10 et 12, la prédiction vaut 10.

Question de compréhension

Quelle modification de max_depth pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

5. Arbre de régression — mise en pratique

Python — API ou principe

```
DecisionTreeRegressor(max_depth=5, random_state=42)
```

- Avantage : Non linéaire et interprétable
- Limite : Instable et extrapole mal
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

Définition

Agréger les prédictions d'arbres entraînés sur des échantillons bootstrap.

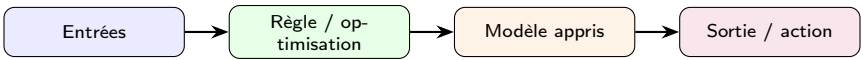
- Problème étudié : Prédiction de la demande export
- Famille : régression supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

6. Forêt aléatoire — régression — principe de fonctionnement

Principe : Bagging et agrégation

Tirer plusieurs échantillons bootstrap des observations.

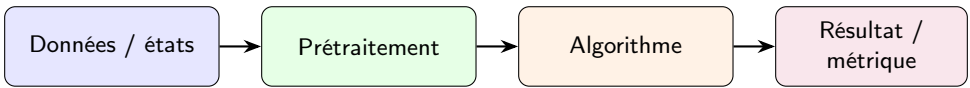
1. Entraîner des arbres de régression avec des sous-ensembles aléatoires de variables.
2. Calculer la prédiction de chaque arbre sur le nouveau point.
3. Moyenner les prédictions pour réduire la variance.



Point de vigilance

Les arbres sont entraînés en parallèle, sans correction séquentielle des erreurs.

6. Forêt aléatoire — régression — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Agréger les prédictions d'arbres entraînés sur des échantillons bootstrap.

6. Forêt aléatoire — régression — équation

Modèle / critère principal

$$\widehat{y}(x) = \frac{1}{B} \sum_{b=1}^{B} T_b(x)$$

Lecture pédagogique

Agréger les prédictions d'arbres entraînés sur des échantillons bootstrap.

Tous les symboles de l'équation

B : nombre d'arbres ; T_b : prédiction du b-ième arbre ; x : nouveau point ; y-hat : moyenne.

Paramètres et hyperparamètres

Hyperparamètre principal : n_estimators / max_depth. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

Exemple numérique

Avec trois arbres donnant 9, 12 et 15 : moyenne=12.

Question de compréhension

Quelle modification de n_estimators / max_depth pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
RandomForestRegressor(n_estimators=80, random_state=42)
```

- Avantage : Robuste et non linéaire
- Limite : Modèle volumineux, extrapolation faible
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

Définition

Construire des arbres qui corrigent progressivement les résidus.

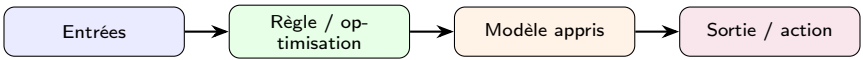
- Problème étudié : Prévision de ventes
- Famille : régression supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

7. Gradient Boosting — régression — principe de fonctionnement

Principe : Ensemble séquentiel additif

Initialiser une prédiction simple, souvent la moyenne de la cible.

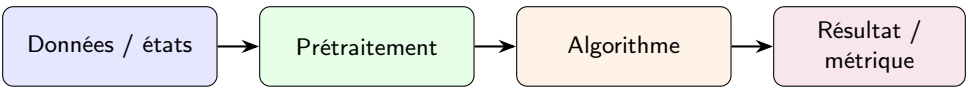
1. Calculer les pseudo-résidus de la fonction de perte.
2. Entraîner un petit arbre à approximer ces corrections.
3. Ajouter l'arbre avec un taux d'apprentissage ; répéter.



Point de vigilance

Le nombre d'arbres et le taux d'apprentissage déterminent ensemble la capacité du modèle.

7. Gradient Boosting — régression — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Construire des arbres qui corrigent progressivement les résidus.

7. Gradient Boosting — régression — équation

Modèle / critère principal

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

Lecture pédagogique

Construire des arbres qui corrigent progressivement les résidus.

7. Gradient Boosting — régression — notations

Tous les symboles de l'équation

F_m : modèle à l'itération m ; h_m : nouvel apprenant faible ; eta : taux d'apprentissage ; x : données.

Paramètres et hyperparamètres

Hyperparamètre principal : learning_rate / max_iter. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

7. Gradient Boosting — régression — exemple calculé

Exemple numérique

Prédiction initiale 10, correction +4 avec eta=0,1 : nouveau score=10,4.

Question de compréhension

Quelle modification de learning_rate / max_iter pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
HistGradientBoostingRegressor(max_iter=80, random_state=42)
```

- Avantage : Puissant sur tableaux
- Limite : Réglage sensible, risque de surapprentissage
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

Plan — 3. Algorithmes de classification

- Présentation des algorithmes de algorithmes de classification.
- Relier intuition, fonctionnement, équations et paramètres.
- Calcul numérique, schéma et exemple Python.
- Discussion : limites, métriques et applications.

8. Régression logistique — intuition

Définition

Transformer un score linéaire en probabilité par une sigmoïde.

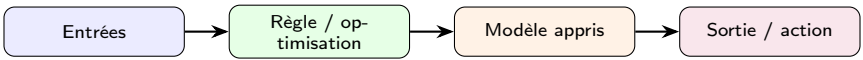
- Problème étudié : Défaut de paiement
- Famille : classification supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

8. Régression logistique — principe de fonctionnement

Principe : Classification probabiliste

Former un score linéaire à partir des caractéristiques.

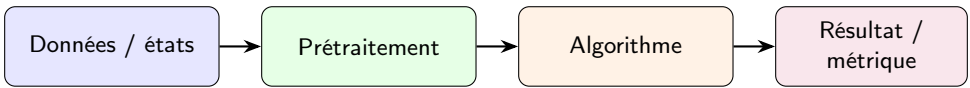
1. Transformer ce score en probabilité par une sigmoïde.
2. Ajuster les coefficients en minimisant la perte logistique.
3. Comparer la probabilité à un seuil adapté au coût des erreurs.



Point de vigilance

La sortie est une probabilité estimée ; le seuil ne fait pas partie de la sigmoïde.

8. Régression logistique — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Transformer un score linéaire en probabilité par une sigmoïde.

8. Régression logistique — équation

Modèle / critère principal

$$p(x) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \boldsymbol{\beta}^\top x$$

Lecture pédagogique

Transformer un score linéaire en probabilité par une sigmoïde.

8. Régression logistique — notations

Tous les symboles de l'équation

p(x) : probabilité de la classe 1 ; sigma : sigmoïde ; z : logit ; beta_0 : biais ; beta : poids ; x : caractéristiques.

Paramètres et hyperparamètres

Hyperparamètre principal : C. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

8. Régression logistique — exemple calculé

Exemple numérique

Pour $z=0$, $p=0{,}5$; pour $z=2$, $p\approx 0{,}881$.

Question de compréhension

Quelle modification de C pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
LogisticRegression(max_iter=1000, random_state=42)
```

- Avantage : Probas interprétables
- Limite : Frontière linéaire sans transformation
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

9. K plus proches voisins — intuition

Définition

Attribuer la classe majoritaire parmi les k observations les plus proches.

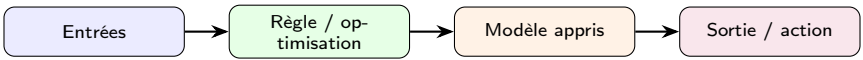
- Problème étudié : Identification de profils clients
- Famille : classification supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

9. K plus proches voisins — principe de fonctionnement

Principe : Vote local

Standardiser les variables lorsque la distance le nécessite.

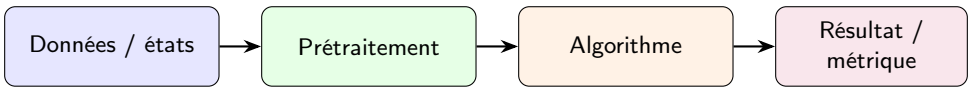
1. Conserver les observations d'entraînement et leurs étiquettes.
2. Mesurer la distance du nouvel exemple à chacun des points.
3. Retenir k voisins et agréger leurs classes par vote.



Point de vigilance

C'est une méthode d'apprentissage paresseux ; la prédiction peut devenir coûteuse.

9. K plus proches voisins — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Attribuer la classe majoritaire parmi les k observations les plus proches.

9. K plus proches voisins — équation

Modèle / critère principal

$$\widehat{y}(x) = \text{mode}\{y_i : i \in N_k(x)\}, \quad d(x,z) = \sqrt{\sum_j (x_j - z_j)^2}$$

Lecture pédagogique

Attribuer la classe majoritaire parmi les k observations les plus proches.

9. K plus proches voisins — notations

Tous les symboles de l'équation

N_k(x) : k voisins ; k : hyperparamètre ; y_i : étiquettes ; d : distance ; x,z : vecteurs de caractéristiques.

Paramètres et hyperparamètres

Hyperparamètre principal : n_neighbors. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

Exemple numérique

Si 3 voisins sont A,A,B, la prédiction majoritaire est A.

Question de compréhension

Quelle modification de n_neighbors pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

9. K plus proches voisins — mise en pratique

Python — API ou principe

```
KNeighborsClassifier(n_neighbors=5)
```

- Avantage : Simple, sans apprentissage paramétrique
- Limite : Sensibilité à l'échelle et à la dimension
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

Définition

Combiner les probabilités conditionnelles sous hypothèse d'indépendance naïve.

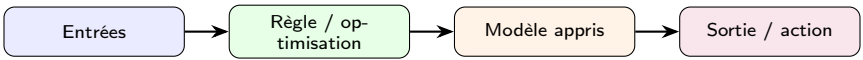
- Problème étudié : Filtrage de courriels commerciaux
- Famille : classification supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

10. Naive Bayes — principe de fonctionnement

Principe : Inférence bayésienne

Estimer la probabilité a priori de chaque classe.

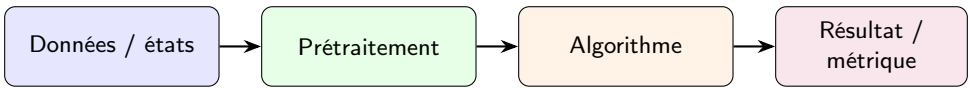
1. Estimer la vraisemblance de chaque variable conditionnellement à la classe.
2. Combiner prior et vraisemblances selon Bayes.
3. Prédire la classe de probabilité a posteriori maximale.



Point de vigilance

L'indépendance conditionnelle est une hypothèse simplificatrice, souvent approximative.

10. Naive Bayes — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Combiner les probabilités conditionnelles sous hypothèse d'indépendance naïve.

10. Naive Bayes — équation

Modèle / critère principal

$$P(C_k \mid x) \propto P(C_k) \prod_{j=1}^{p} P(x_j \mid C_k)$$

Lecture pédagogique

Combiner les probabilités conditionnelles sous hypothèse d'indépendance naïve.

10. Naive Bayes — notations

Tous les symboles de l'équation

C_k : classe k ; x_j : attribut j ; P(C_k) : a priori ; P(x_j|C_k) : vraisemblance conditionnelle.

Paramètres et hyperparamètres

Hyperparamètre principal : var_smoothing. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

10. Naive Bayes — exemple calculé

Exemple numérique

Si P(spam)=0,2 et deux indices probables, le produit doit être normalisé entre classes.

Question de compréhension

Quelle modification de var_smoothing pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
GaussianNB()
```

- Avantage : Rapide avec peu de données
- Limite : Indépendance rarement vraie
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

11. Arbre de décision — intuition

Définition

Choisir des questions successives qui réduisent l'impureté.

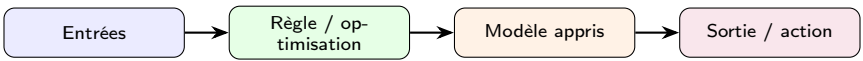
- Problème étudié : Qualification d'un marché export
- Famille : classification supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

11. Arbre de décision — principe de fonctionnement

Principe : Règles de séparation

Mesurer l'impureté des étiquettes au nœud courant.

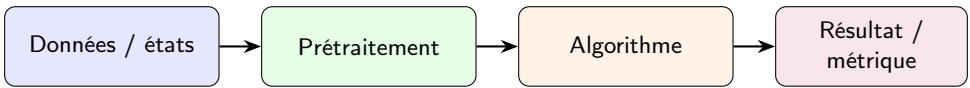
1. Tester des divisions selon une variable et un seuil.
2. Choisir la division diminuant le plus l'impureté.
3. Répéter jusqu'au critère d'arrêt puis attribuer une classe à chaque feuille.



Point de vigilance

Un arbre profond est facilement interprétable localement mais sensible au surapprentissage.

11. Arbre de décision — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Choisir des questions successives qui réduisent l'impureté.

11. Arbre de décision — équation

Modèle / critère principal

$$G(t) = 1 - \sum_{k=1}^{K} p_{k,t}^2$$

Lecture pédagogique

Choisir des questions successives qui réduisent l'impureté.

11. Arbre de décision — notations

Tous les symboles de l'équation

G(t) : impureté de Gini au nœud t ; K : nombre de classes ; p_{k,t} : proportion de la classe k au nœud t.

Paramètres et hyperparamètres

Hyperparamètre principal : max_depth / min_samples_leaf. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

11. Arbre de décision — exemple calculé

Exemple numérique

Nœud à 50 % / 50 % : G=1-0,25-0,25=0,5.

Question de compréhension

Quelle modification de max_depth / min_samples_leaf pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
DecisionTreeClassifier(max_depth=4, random_state=42)
```

- Avantage : Règles lisibles
- Limite : Variabilité des frontières, surapprentissage
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

Définition

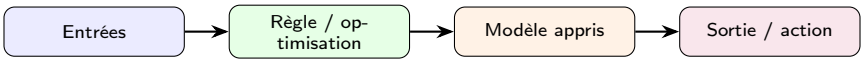
Voter à partir de plusieurs arbres décorrelés.

- Problème étudié : Détection de risque commercial
- Famille : classification supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

Principe : Vote d'ensemble

Former plusieurs échantillons bootstrap.

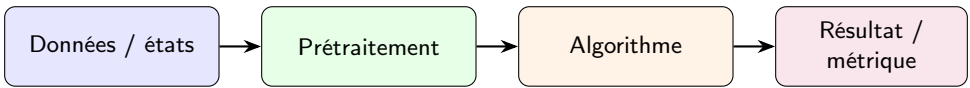
1. Construire des arbres avec sous-échantillonnage de variables.
2. Faire produire une classe ou des probabilités par chaque arbre.
3. Agrèger les votes ou les probabilités des arbres.



Point de vigilance

La diversité des arbres est essentielle au gain de généralisation.

12. Forêt aléatoire — classification — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Voter à partir de plusieurs arbres décorrelés.

12. Forêt aléatoire — classification — équation

Modèle / critère principal

$$\widehat{y}(x) = \text{mode}\{T_1(x), \ldots, T_B(x)\}$$

Lecture pédagogique

Voter à partir de plusieurs arbres décorrelés.

Tous les symboles de l'équation

B : nombre d'arbres ; T_b(x) : classe proposée par l'arbre b ; mode : vote majoritaire ; x : entrée.

Paramètres et hyperparamètres

Hyperparamètre principal : n_estimators / max_features. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

12. Forêt aléatoire — classification — exemple calculé

Exemple numérique

Votes A, B, A, A, B donnent A.

Question de compréhension

Quelle modification de n_estimators / max_features pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

12. Forêt aléatoire — classification — mise en pratique

Python — API ou principe

```
RandomForestClassifier(n_estimators=80, random_state=42)
```

- Avantage : Bonne précision sur tableaux
- Limite : Explication individuelle plus difficile
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

13. Machines à vecteurs de support — intuition

Définition

Chercher l'hyperplan séparateur à marge maximale.

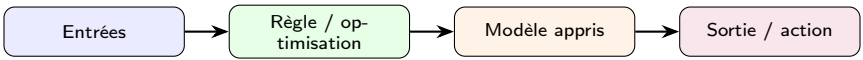
- Problème étudié : Classification de risques rares
- Famille : classification supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

13. Machines à vecteurs de support — principe de fonctionnement

Principe : Marge maximale

Transformer éventuellement les caractéristiques avec un noyau.

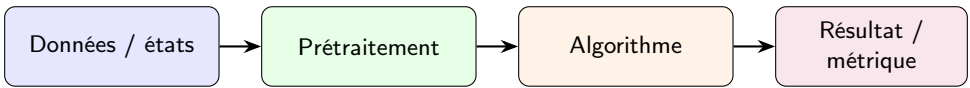
1. Chercher l'hyperplan séparant les classes avec une marge élevée.
2. Pénaliser les points qui violent la marge selon le paramètre C.
3. Prédire par le signe de la fonction de décision.



Point de vigilance

Le noyau RBF permet des frontières non linéaires ; standardisation importante.

13. Machines à vecteurs de support — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Chercher l'hyperplan séparateur à marge maximale.

13. Machines à vecteurs de support — équation

Modèle / critère principal

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i, \quad y_i(w^\top x_i + b) \geq 1 - \xi_i$$

Lecture pédagogique

Chercher l'hyperplan séparateur à marge maximale.

13. Machines à vecteurs de support — notations

Tous les symboles de l'équation

w : normale de l'hyperplan ; b : biais ; xi_i : violation de marge >=0 ; C : pénalisation ; y_i∈{-1,+1} ; x_i : observation.

Paramètres et hyperparamètres

Hyperparamètre principal : C / gamma / kernel. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

Exemple numérique

Avec deux séparateurs corrects, préférer celui dont la marge géométrique $2/||w||$ est la plus large.

Question de compréhension

Quelle modification de C / gamma / kernel pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
SVC(C=1.0, kernel="rbf", probability=True, random_state=42)
```

- Avantage : Efficace en moyenne dimension
- Limite : Coûteux sur grands jeux et sensible au noyau
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

14. Gradient Boosting — classification — intuition

Définition

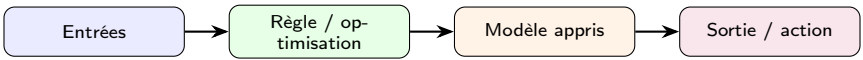
Additionner des arbres correctifs sur les scores de classe.

- Problème étudié : Prévision de souscription
- Famille : classification supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

Principe : Boosting de perte logistique

Initialiser des scores de classe simples.

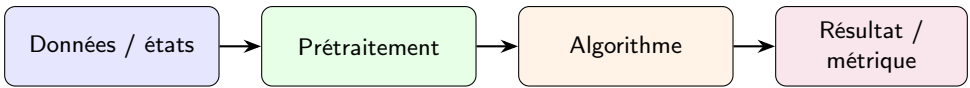
1. Calculer les corrections liées au gradient de la perte logistique.
2. Entraîner un faible arbre sur ces corrections.
3. Mettre à jour les scores, puis en déduire des probabilités.



Point de vigilance

Les arbres se construisent séquentiellement : ce n'est pas une forêt aléatoire.

14. Gradient Boosting — classification — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Additionner des arbres correctifs sur les scores de classe.

14. Gradient Boosting — classification — équation

Modèle / critère principal

$$F_m(x) = F_{m-1}(x) + \eta h_m(x), \quad p(x) = \sigma(F_M(x))$$

Lecture pédagogique

Additionner des arbres correctifs sur les scores de classe.

14. Gradient Boosting — classification — notations

Tous les symboles de l'équation

F_m : score cumulé ; h_m : arbre ajouté ; eta : pas ; M : nombre d'étapes ; sigma : sigmoïde ; p : probabilité binaire.

Paramètres et hyperparamètres

Hyperparamètre principal : learning_rate / max_iter. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

14. Gradient Boosting — classification — exemple calculé

Exemple numérique

Si score initial=0 et correction=2 avec eta=0,2 : score=0,4 puis p≈0,599.

Question de compréhension

Quelle modification de learning_rate / max_iter pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

14. Gradient Boosting — classification — mise en pratique

Python — API ou principe

```
HistGradientBoostingClassifier(max_iter=80, random_state=42)
```

- Avantage : Très compétitif sur tableaux
- Limite : Risque de surentraînement si trop d'arbres
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

15. AdaBoost — intuition

Définition

Accroître les poids des exemples mal classés pour entraîner des apprenants successifs.

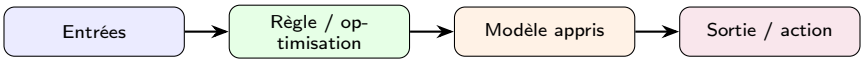
- Problème étudié : Classification de prospects
- Famille : classification supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

15. AdaBoost — principe de fonctionnement

Principe : Repondération séquentielle

Attribuer un poids identique à chaque observation.

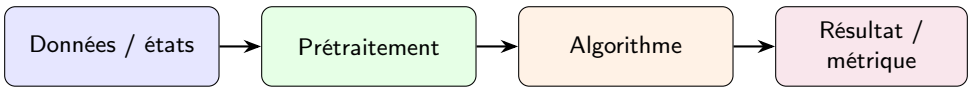
1. Entraîner un classificateur simple, souvent une souche d'arbre.
2. Augmenter les poids des observations mal classées.
3. Répéter et combiner les classificateurs par vote pondéré.



Point de vigilance

Très sensible aux étiquettes bruitées et aux valeurs difficiles à classifier.

15. AdaBoost — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Accroître les poids des exemples mal classés pour entraîner des apprenants successifs.

15. AdaBoost — équation

Modèle / critère principal

$$F(x) = \sum_{m=1}^{M} \alpha_m h_m(x)$$

Lecture pédagogique

Accroître les poids des exemples mal classés pour entraîner des apprenants successifs.

15. AdaBoost — notations

Tous les symboles de l'équation

M : nombre d'apprenants ; h_m : score ou prédiction du modèle m ; alpha_m : poids de contribution ; F : agrégation.

Paramètres et hyperparamètres

Hyperparamètre principal : n_estimators / learning_rate. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

15. AdaBoost — exemple calculé

Exemple numérique

Deux classificateurs de poids 0,7 et 0,3 : le premier influence davantage la décision.

Question de compréhension

Quelle modification de n_estimators / learning_rate pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

15. AdaBoost — mise en pratique

Python — API ou principe

```
AdaBoostClassifier(n_estimators=70, random_state=42)
```

- Avantage : Facile avec arbres faibles
- Limite : Sensible à des étiquettes très bruitées
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

16. XGBoost — intuition

Définition

Implémentation optimisée du gradient boosting d'arbres avec pénalités.

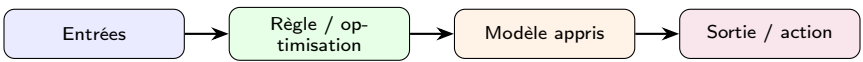
- Problème étudié : Prévision de désengagement client
- Famille : classification supervisée
- On distingue les entrées, la règle apprise et la sortie attendue.

16. XGBoost — principe de fonctionnement

Principe : Boosting de second ordre

Initialiser les scores du modèle.

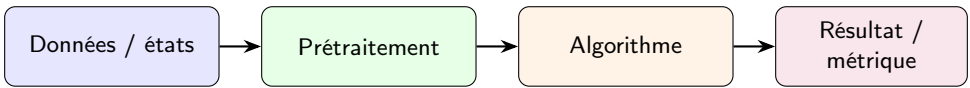
1. Calculer gradients et courbures de la fonction objective.
2. Construire un arbre optimisant le gain régularisé des divisions.
3. Ajouter l'arbre au modèle avec réduction de pas et répétition.



Point de vigilance

Utilise des optimisations et pénalités explicites, dont la régularisation des feuilles.

16. XGBoost — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Implémentation optimisée du gradient boosting d'arbres avec pénalités.

16. XGBoost — équation

Modèle / critère principal

$$\mathcal{L} = \sum_{i=1}^{n} \ell(y_i, \widehat{y}_i) + \sum_{m=1}^{M} \Omega(f_m)$$

Lecture pédagogique

Implémentation optimisée du gradient boosting d'arbres avec pénalités.

16. XGBoost — notations

Tous les symboles de l'équation

ell : perte par observation ; y_i : vérité ; y-hat_i : prédiction ; Omega : pénalité de complexité des arbres f_m ; M : nombre d'arbres.

Paramètres et hyperparamètres

Hyperparamètre principal : max_depth / learning_rate / n_estimators. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

16. XGBoost — exemple calculé

Exemple numérique

Ajouter un arbre n'est pertinent que si le gain de perte dépasse sa pénalité.

Question de compréhension

Quelle modification de max_depth / learning_rate / n_estimators pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

16. XGBoost — mise en pratique

Python — API ou principe

```
XGBClassifier(n_estimators=80, max_depth=3, random_state=42, eval_metric="logloss")
```

- Avantage : Efficace et flexible
- Limite : Dépendance supplémentaire, tuning
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

Plan — 4. Partitionnement et regroupement

- Présentation des algorithmes de partitionnement et regroupement.
- Relier intuition, fonctionnement, équations et paramètres.
- Calcul numérique, schéma et exemple Python.
- Discussion : limites, métriques et applications.

17. K-means — intuition

Définition

Alterner affectation aux centroïdes et mise à jour des moyennes.

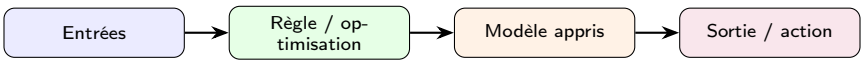
- Problème étudié : Segmentation des partenaires commerciaux
- Famille : clustering
- On distingue les entrées, la règle apprise et la sortie attendue.

17. K-means — principe de fonctionnement

Principe : Affectation et mise à jour

Choisir k et initialiser k centroïdes (souvent k-means++).

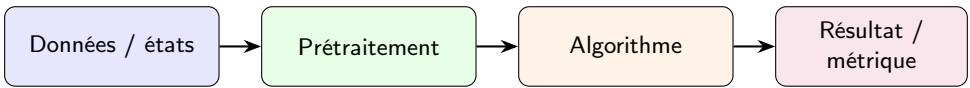
1. Affecter chaque observation au centroïde le plus proche.
2. Recalculer chaque centroïde comme moyenne de son groupe.
3. Répéter affectation et mise à jour jusqu'à stabilisation.



Point de vigilance

Minimise l'inertie intra-groupe ; suppose des groupes compatibles avec la distance choisie.

17. K-means — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Alterner affectation aux centroïdes et mise à jour des moyennes.

17. K-means — équation

Modèle / critère principal

$$J = \sum_{i=1}^{n} \min_{1 \leq k \leq K} \|x_i - \mu_k\|_2^2$$

Lecture pédagogique

Alterner affectation aux centroïdes et mise à jour des moyennes.

17. K-means — notations

Tous les symboles de l'équation

n : observations ; K : nombre de groupes ; x_i : vecteur i ; mu_k : centroïde k ; $||.||^2$: distance quadratique.

Paramètres et hyperparamètres

Hyperparamètre principal : n_clusters. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

17. K-means — exemple calculé

Exemple numérique

Pour points (0,0) et (2,0) dans un groupe : centroïde (1,0).

Question de compréhension

Quelle modification de n_clusters pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

Python — API ou principe

```
KMeans(n_clusters=3, n_init=10, random_state=42)
```

- Avantage : Très rapide
- Limite : Choix de K et sensibilité aux outliers
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

18. Classification hiérarchique — intuition

Définition

Fusionner progressivement les groupes les plus proches.

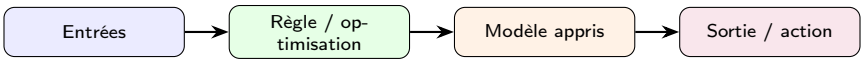
- Problème étudié : Typologie de marchés par similarité
- Famille : clustering
- On distingue les entrées, la règle apprise et la sortie attendue.

18. Classification hiérarchique — principe de fonctionnement

Principe : Fusions successives

Considérer d'abord chaque observation comme un groupe.

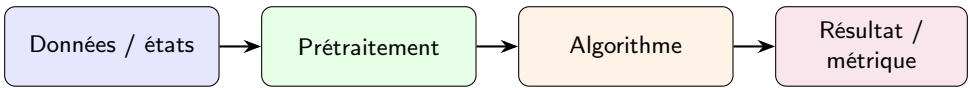
1. Mesurer la distance entre groupes selon un critère de liaison.
2. Fusionner les deux groupes les plus proches.
3. Répéter pour construire un dendrogramme ; couper à une hauteur choisie.



Point de vigilance

La liaison de Ward cherche à minimiser l'augmentation d'inertie lors de la fusion.

18. Classification hiérarchique — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Fusionner progressivement les groupes les plus proches.

18. Classification hiérarchique — équation

Modèle / critère principal

$$d(A,B) = \min_{x \in A, z \in B} d(x,z) \quad \text{(liaison simple)}$$

Lecture pédagogique

Fusionner progressivement les groupes les plus proches.

18. Classification hiérarchique — notations

Tous les symboles de l'équation

A,B : clusters ; x,z : observations ; d : distance entre points ; d(A,B) : distance entre groupes ; autres liaisons possibles.

Paramètres et hyperparamètres

Hyperparamètre principal : linkage / n_clusters. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

18. Classification hiérarchique — exemple calculé

Exemple numérique

Si A={0,1}, B={3,4}, distance simple=2.

Question de compréhension

Quelle modification de linkage / n_clusters pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

18. Classification hiérarchique — mise en pratique

Python — API ou principe

```
AgglomerativeClustering(n_clusters=3)
```

- Avantage : Dendrogramme explicatif
- Limite : Coût mémoire sur grands jeux
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

19. DBSCAN — intuition

Définition

Définir des groupes denses et marquer le bruit.

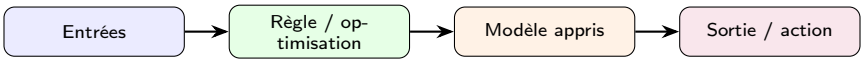
- Problème étudié : Détection de flux commerciaux isolés
- Famille : clustering
- On distingue les entrées, la règle apprise et la sortie attendue.

19. DBSCAN — principe de fonctionnement

Principe : Regroupement par densité

Fixer un rayon eps et un nombre minimal de points.

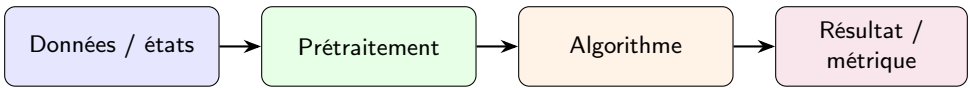
1. Identifier les points centraux ayant assez de voisins.
2. Étendre chaque groupe depuis ces points par densité-connectivité.
3. Laisser comme bruit les points non atteints.



Point de vigilance

Ne nécessite pas le nombre de clusters mais réagit fortement à eps et à l'échelle.

19. DBSCAN — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Définir des groupes denses et marquer le bruit.

19. DBSCAN — équation

Modèle / critère principal

$$N_\varepsilon(x) = \{z : d(x,z) \leq \varepsilon\}, \quad |N_\varepsilon(x)| \geq m$$

Lecture pédagogique

Définir des groupes denses et marquer le bruit.

19. DBSCAN — notations

Tous les symboles de l'équation

epsilon : rayon de voisinage ; N_epsilon(x) : points proches ; m : min_samples ; d : distance choisie ; x : observation.

Paramètres et hyperparamètres

Hyperparamètre principal : eps / min_samples. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

19. DBSCAN — exemple calculé

Exemple numérique

Si eps=1 et 5 voisins pour min_samples=4, x est un point cœur.

Question de compréhension

Quelle modification de eps / min_samples pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

19. DBSCAN — mise en pratique

Python — API ou principe

```
DBSCAN(eps=0.8, min_samples=5)
```

- Avantage : Formes arbitraires et bruit
- Limite : Paramétrage difficile à densités hétérogènes
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

20. Mélanges gaussiens — intuition

Définition

Modéliser les données par une somme de distributions normales.

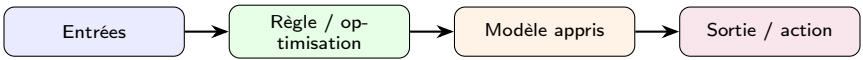
- Problème étudié : Profils de clients probabilistes
- Famille : clustering
- On distingue les entrées, la règle apprise et la sortie attendue.

20. Mélanges gaussiens — principe de fonctionnement

Principe : Algorithme EM

Fixer un nombre de composantes gaussiennes et initialiser leurs paramètres.

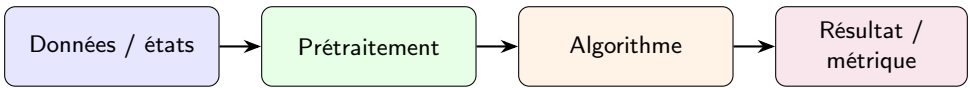
1. Étape E : estimer la responsabilité de chaque composante pour chaque observation.
2. Étape M : recalculer poids, moyennes et covariances.
3. Répéter jusqu'à convergence puis attribuer des probabilités de composante.



Point de vigilance

Les appartenances sont souples ; le modèle suppose un mélange de densités gaussiennes.

20. Mélanges gaussiens — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Modéliser les données par une somme de distributions normales.

20. Mélanges gaussiens — équation

Modèle / critère principal

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

Lecture pédagogique

Modéliser les données par une somme de distributions normales.

20. Mélanges gaussiens — notations

Tous les symboles de l'équation

K : composantes ; pi_k : proportion du groupe k, somme=1 ; mu_k : moyenne ; Sigma_k : covariance ; N : densité normale.

Paramètres et hyperparamètres

Hyperparamètre principal : n_components / covariance_type. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

20. Mélanges gaussiens — exemple calculé

Exemple numérique

Si pi=(0,3 ;0,7), le second groupe pèse 70 % dans le mélange.

Question de compréhension

Quelle modification de n_components / covariance_type pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

20. Mélanges gaussiens — mise en pratique

Python — API ou principe

```
GaussianMixture(n_components=3, random_state=42)
```

- Avantage : Assignation probabiliste
- Limite : Hypothèse de mélange gaussien
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

21. Analyse en composantes principales — intuition

Définition

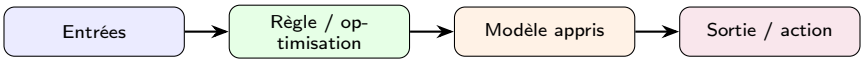
Projeter sur les directions maximisant la variance.

- Problème étudié : Visualisation de ratios financiers
- Famille : clustering
- On distingue les entrées, la règle apprise et la sortie attendue.

Principe : Projection orthogonale

Centrer, et éventuellement réduire, les colonnes numériques.

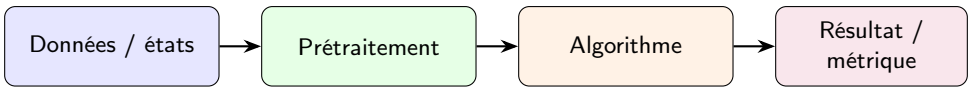
1. Calculer les directions de plus grande variance via SVD ou covariance.
2. Trier les axes par variance expliquée décroissante.
3. Projeter les observations sur les premières composantes.



Point de vigilance

L'ACP est linéaire et non supervisée ; la plus grande variance n'est pas toujours prédictive.

21. Analyse en composantes principales — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Projeter sur les directions maximisant la variance.

21. Analyse en composantes principales — équation

Modèle / critère principal

$$Z = X_c W_r, \quad X_c = X - 1\bar{x}^\top$$

Lecture pédagogique

Projeter sur les directions maximisant la variance.

21. Analyse en composantes principales — notations

Tous les symboles de l'équation

X : données n x p ; X_c : données centrées ; bar x : moyennes des colonnes ; W_r : p x r vecteurs propres ; Z : projection n x r.

Paramètres et hyperparamètres

Hyperparamètre principal : n_components. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

21. Analyse en composantes principales — exemple calculé

Exemple numérique

Une projection de 5 variables vers 2 composantes donne Z de taille n x 2.

Question de compréhension

Quelle modification de n_components pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

21. Analyse en composantes principales — mise en pratique

Python — API ou principe

```
PCA(n_components=2)
```

- Avantage : Compression et débruitage
- Limite : Transformation linéaire et sensible aux échelles
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

22. t-SNE — intuition

Définition

Préserver les voisinages locaux dans une carte de basse dimension.

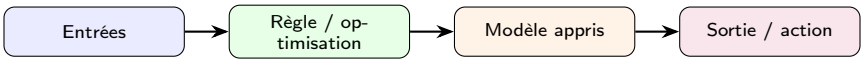
- Problème étudié : Cartographie de profils produits
- Famille : clustering
- On distingue les entrées, la règle apprise et la sortie attendue.

22. t-SNE — principe de fonctionnement

Principe : Préservation locale

Calculer les voisinages probabilistes dans l'espace d'origine.

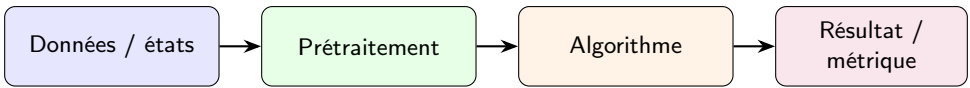
1. Initialiser les points dans un espace 2D ou 3D.
2. Ajuster leurs positions pour conserver les voisinages locaux.
3. Visualiser les structures locales sans surinterpréter les distances lointaines.



Point de vigilance

La perplexité et la graine influencent fortement les projections ; pas une méthode de prédiction.

22. t-SNE — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Préserver les voisinages locaux dans une carte de basse dimension.

22. t-SNE — équation

Modèle / critère principal

$$\mathrm{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Lecture pédagogique

Préserver les voisinages locaux dans une carte de basse dimension.

22. t-SNE — notations

Tous les symboles de l'équation

p_ij : affinité entre points dans l'espace initial ; q_ij : affinité projetée ; KL : divergence ; i,j : paires de points.

Paramètres et hyperparamètres

Hyperparamètre principal : perplexity / learning_rate. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

22. t-SNE — exemple calculé

Exemple numérique

Deux points voisins dans l'espace initial doivent avoir une affinité projetée élevée.

Question de compréhension

Quelle modification de perplexity / learning_rate pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

22. t-SNE — mise en pratique

Python — API ou principe

```
TSNE(n_components=2, perplexity=20, random_state=42)
```

- Avantage : Visualisation locale expressive
- Limite : Distances globales non fiables, coût
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

23. UMAP — intuition

Définition

Construire une représentation locale fondée sur un graphe des voisins.

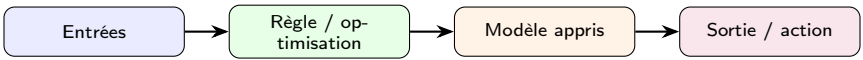
- Problème étudié : Exploration de structures complexes
- Famille : clustering
- On distingue les entrées, la règle apprise et la sortie attendue.

23. UMAP — principe de fonctionnement

Principe : Graphe de voisinage

Construire un graphe des plus proches voisins.

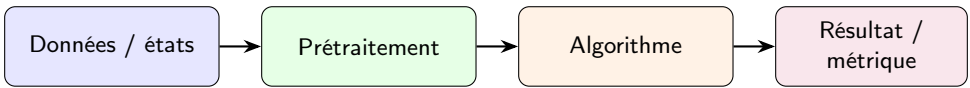
1. Estimer une structure topologique locale à partir des distances.
2. Optimiser une représentation de faible dimension qui préserve ce graphe.
3. Comparer plusieurs graines et paramètres de voisinage.



Point de vigilance

La représentation dépend des paramètres et n'assure pas une géométrie globale exacte.

23. UMAP — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Construire une représentation locale fondée sur un graphe des voisins.

23. UMAP — équation

Modèle / critère principal

$$\mathcal{L} = -\sum_{ij}[p_{ij}\log q_{ij} + (1-p_{ij})\log(1-q_{ij})]$$

Lecture pédagogique

Construire une représentation locale fondée sur un graphe des voisins.

23. UMAP — notations

Tous les symboles de l'équation

p_ij : poids des arêtes dans le graphe haut dimensionnel ; q_ij : poids projeté ; L : objectif de proximité ; indices i,j : paires de points.

Paramètres et hyperparamètres

Hyperparamètre principal : n_neighbors / min_dist. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

23. UMAP — exemple calculé

Exemple numérique

Des points fortement reliés doivent rester voisins dans la projection.

Question de compréhension

Quelle modification de n_neighbors / min_dist pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

23. UMAP — mise en pratique

Python — API ou principe

```
umap.UMAP(n_neighbors=15, random_state=42)
```

- Avantage : Rapide pour grands jeux
- Limite : Distorsions possibles, paquet additionnel
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

24. Isolation Forest — intuition

Définition

Repérer les observations isolables par des partitions aléatoires courtes.

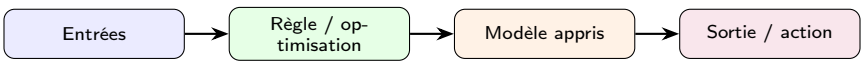
- Problème étudié : Anomalies dans les opérations commerciales
- Famille : clustering
- On distingue les entrées, la règle apprise et la sortie attendue.

24. Isolation Forest — principe de fonctionnement

Principe : Isolement aléatoire

Tirer des sous-échantillons aléatoires.

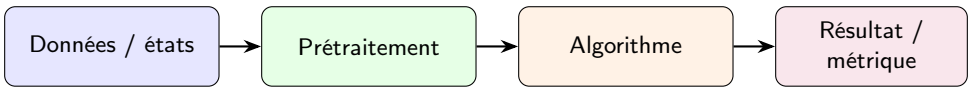
1. Créer des arbres avec variables et seuils de séparation aléatoires.
2. Mesurer la longueur moyenne des chemins nécessaires pour isoler chaque point.
3. Attribuer un score d'anomalie élevé aux points isolés rapidement.



Point de vigilance

Un point facile à isoler est statistiquement atypique, pas nécessairement frauduleux.

24. Isolation Forest — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Repérer les observations isolables par des partitions aléatoires courtes.

24. Isolation Forest — équation

Modèle / critère principal

$$s(x, n) = 2^{-\frac{\mathbb{E}[h(x)]}{c(n)}}$$

Lecture pédagogique

Repérer les observations isolables par des partitions aléatoires courtes.

24. Isolation Forest — notations

Tous les symboles de l'équation

h(x) : longueur du chemin pour isoler x ; E : moyenne entre arbres ; c(n) : normalisateur selon la taille n ; s : score d'anomalie.

Paramètres et hyperparamètres

Hyperparamètre principal : contamination / n_estimators. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

24. Isolation Forest — exemple calculé

Exemple numérique

Un point isolé après 2 divisions paraît plus atypique qu'un point isolé après 10.

Question de compréhension

Quelle modification de contamination / n_estimators pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

24. Isolation Forest — mise en pratique

Python — API ou principe

```
IsolationForest(contamination=0.05, random_state=42)
```

- Avantage : Détection sans cible
- Limite : Dépend de la contamination et des variables
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

Plan — 7. Apprentissage par renforcement

- Présentation des algorithmes de apprentissage par renforcement.
- Relier intuition, fonctionnement, équations et paramètres.
- Calcul numérique, schéma et exemple Python.
- Discussion : limites, métriques et applications.

25. Bandits manchots — intuition

Définition

Choisir une action pour équilibrer exploration et exploitation.

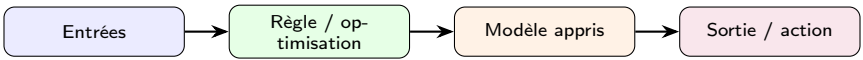
- Problème étudié : Choix de campagne commerciale
- Famille : renforcement
- On distingue les entrées, la règle apprise et la sortie attendue.

25. Bandits manchots — principe de fonctionnement

Principe : Exploration et exploitation

Définir plusieurs actions aux récompenses inconnues.

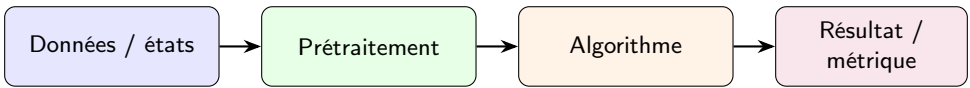
1. Sélectionner un bras selon une stratégie d'exploration.
2. Observer la récompense et actualiser l'estimation du bras.
3. Répéter afin d'augmenter la récompense cumulée.



Point de vigilance

Un bandit ne possède pas de dynamique d'états comme un MDP général.

25. Bandits manchots — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Choisir une action pour équilibrer exploration et exploitation.

25. Bandits manchots — équation

Modèle / critère principal

$$Q_t(a) = \frac{1}{N_t(a)} \sum_{j=1}^{N_t(a)} r_j(a)$$

Lecture pédagogique

Choisir une action pour équilibrer exploration et exploitation.

25. Bandits manchots — notations

Tous les symboles de l'équation

Q_t(a) : estimation moyenne du gain ; N_t(a) : nombre de choix de l'action a ; r_j(a) : récompense observée au j-ième choix.

Paramètres et hyperparamètres

Hyperparamètre principal : horizon. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

25. Bandits manchots — exemple calculé

Exemple numérique

Bras A : gains 1,0,1 -> moyenne 2/3.

Question de compréhension

Quelle modification de horizon pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

25. Bandits manchots — mise en pratique

Python — API ou principe

`Simulation multi-bras en NumPy`

- Avantage : Simple décision séquentielle
- Limite : Pas d'état temporel riche
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

26. Epsilon-greedy — intuition

Définition

Explorer au hasard avec probabilité epsilon, sinon choisir l'action jugée meilleure.

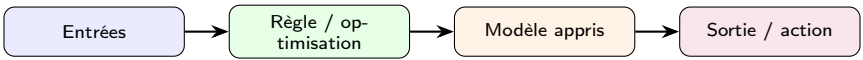
- Problème étudié : Tests A/B adaptatifs
- Famille : renforcement
- On distingue les entrées, la règle apprise et la sortie attendue.

26. Epsilon-greedy — principe de fonctionnement

Principe : Politique de sélection

Pour chaque action, maintenir une valeur moyenne estimée.

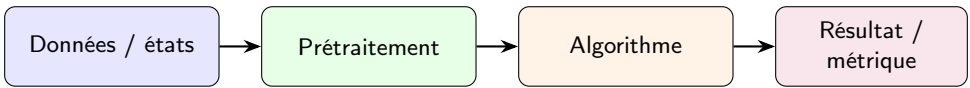
1. Avec probabilité epsilon, choisir une action aléatoire.
2. Sinon, choisir l'action estimée la meilleure.
3. Actualiser l'estimation de l'action jouée avec la récompense.



Point de vigilance

Un epsilon constant maintient une exploration permanente ; sa décroissance est un choix de politique.

26. Epsilon-greedy — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Explorer au hasard avec probabilité epsilon, sinon choisir l'action jugée meilleure.

26. Epsilon-greedy — équation

Modèle / critère principal

$$a_t = \begin{cases} \text{aléatoire}, & \text{avec probabilité } \varepsilon \\ \arg\max_a Q_t(a), & \text{sinon} \end{cases}$$

Lecture pédagogique

Explorer au hasard avec probabilité epsilon, sinon choisir l'action jugée meilleure.

26. Epsilon-greedy — notations

Tous les symboles de l'équation

a_t : action choisie ; epsilon : taux d'exploration (0..1) ; Q_t(a) : valeur estimée de a ; argmax : action au gain maximal.

Paramètres et hyperparamètres

Hyperparamètre principal : epsilon. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

26. Epsilon-greedy — exemple calculé

Exemple numérique

Pour epsilon=0,1 : exploration en moyenne sur 10 % des décisions.

Question de compréhension

Quelle modification de epsilon pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

26. Epsilon-greedy — mise en pratique

Python — API ou principe

```
Simulation bandit + exploration
```

- Avantage : Compromis transparent
- Limite : Exploration non ciblée
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

27. Q-learning — intuition

Définition

Mettre à jour la valeur optimale estimée avec la meilleure action future.

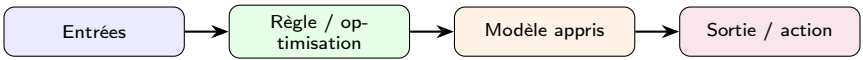
- Problème étudié : Navigation d'un agent logistique
- Famille : renforcement
- On distingue les entrées, la règle apprise et la sortie attendue.

27. Q-learning — principe de fonctionnement

Principe : Mise à jour hors politique

Initialiser une table Q pour les couples état-action.

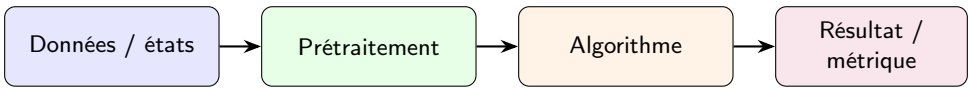
1. Choisir une action exploratoire ou gloutonne selon la politique comportementale.
2. Observer récompense, nouvel état et éventuelle terminaison.
3. Mettre à jour Q avec la meilleure valeur future estimée puis répéter.



Point de vigilance

Q-learning est hors politique : la cible utilise la meilleure action future, même si elle n'est pas choisie.

27. Q-learning — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Mettre à jour la valeur optimale estimée avec la meilleure action future.

27. Q-learning — équation

Modèle / critère principal

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a\prime} Q(s\prime, a\prime) - Q(s,a)]$$

Lecture pédagogique

Mettre à jour la valeur optimale estimée avec la meilleure action future.

27. Q-learning — notations

Tous les symboles de l'équation

s : état actuel ; a : action ; r : récompense immédiate ; s-prime : état suivant ; alpha : pas d'apprentissage ; gamma : actualisation ; Q : valeur action-état.

Paramètres et hyperparamètres

Hyperparamètre principal : alpha / gamma / epsilon. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

27. Q-learning — exemple calculé

Exemple numérique

Q=2, r=1, gamma=0,9, maxQsuiv=4, alpha=0,5 => Qnouveau=3,3.

Question de compréhension

Quelle modification de alpha / gamma / epsilon pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

27. Q-learning — mise en pratique

Python — API ou principe

`Q-learning manuel FrozenLake`

- Avantage : Apprend une politique optimale tabulaire
- Limite : Coûteux avec états continus
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

28. SARSA — intuition

Définition

Mettre à jour les valeurs à partir de l'action réellement sélectionnée ensuite.

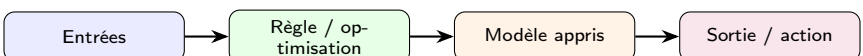
- Problème étudié : Navigation à risque
- Famille : renforcement
- On distingue les entrées, la règle apprise et la sortie attendue.

28. SARSA — principe de fonctionnement

Principe : Mise à jour sur politique

Initialiser les valeurs Q et sélectionner une action a.

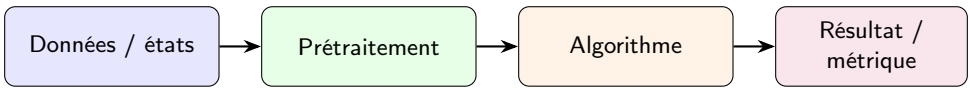
1. Exécuter a puis observer récompense et nouvel état.
2. Sélectionner réellement la prochaine action a' selon la politique courante.
3. Mettre à jour Q avec la valeur Q(s',a') puis poursuivre.



Point de vigilance

SARSA est sur politique : la cible tient compte des actions réellement explorées.

28. SARSA — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Mettre à jour les valeurs à partir de l'action réellement sélectionnée ensuite.

28. SARSA — équation

Modèle / critère principal

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s\prime, a\prime) - Q(s,a)]$$

Lecture pédagogique

Mettre à jour les valeurs à partir de l'action réellement sélectionnée ensuite.

28. SARSA — notations

Tous les symboles de l'équation

Q : valeur action-état ; s,a : état et action ; s-prime,a-prime : état et action suivants réellement choisis ; alpha : pas ; gamma : actualisation.

Paramètres et hyperparamètres

Hyperparamètre principal : alpha / gamma / epsilon. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

28. SARSA — exemple calculé

Exemple numérique

Q=2, r=1, gamma=0,9, Qsuiv=3, alpha=0,5 => Qnouveau=2,85.

Question de compréhension

Quelle modification de alpha / gamma / epsilon pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

28. SARSA — mise en pratique

Python — API ou principe

`SARSA manuel FrozenLake`

- Avantage : Prend en compte politique exploratoire
- Limite : Dépend de la politique suivie
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

29. Deep Q-Network (DQN) — intuition

Définition

Approximer la fonction Q par un réseau neuronal et un réseau cible.

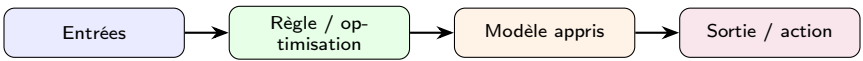
- Problème étudié : Contrôle séquentiel à grands états
- Famille : renforcement
- On distingue les entrées, la règle apprise et la sortie attendue.

29. Deep Q-Network (DQN) — principe de fonctionnement

Principe : Approximation neuronale des valeurs

Approximer les valeurs Q(s,a) par un réseau neuronal.

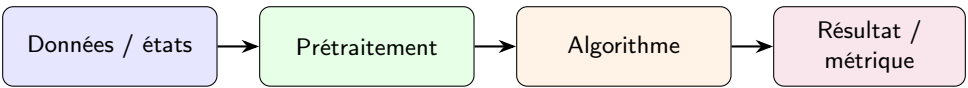
1. Sélectionner des actions par epsilon-greedy et stocker les transitions.
2. Échantillonner des mini-lots dans le tampon de rejeu.
3. Minimiser l'erreur de Bellman avec un réseau cible mis à jour périodiquement.



Point de vigilance

Le rejeu d'expérience et le réseau cible améliorent la stabilité sans la garantir.

29. Deep Q-Network (DQN) — étapes



À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Approximer la fonction Q par un réseau neuronal et un réseau cible.

29. Deep Q-Network (DQN) — équation

Modèle / critère principal

$$y = r + \gamma(1 - d) \max_{a\prime} Q_{\theta^-}(s\prime, a\prime)$$

Lecture pédagogique

Approximer la fonction Q par un réseau neuronal et un réseau cible.

29. Deep Q-Network (DQN) — notations

Tous les symboles de l'équation

y : cible d'entraînement ; r : récompense ; gamma : actualisation ; d : indicateur terminal (0/1) ; theta-moins : poids du réseau cible ; s-prime : état suivant.

Paramètres et hyperparamètres

Hyperparamètre principal : replay / target update. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

29. Deep Q-Network (DQN) — exemple calculé

Exemple numérique

r=1, gamma=0,9, d=0, maxQ=2 => cible y=2,8.

Question de compréhension

Quelle modification de replay / target update pourrait changer la complexité, la stabilité ou l'exploration ? Justifier.

29. Deep Q-Network (DQN) — mise en pratique

Python — API ou principe

```
Aperçu conceptuel (pas DQN complet)
```

- Avantage : États haute dimension
- Limite : Instabilité et besoin important de données
- Travail : entraîner, comparer une référence et visualiser une métrique.
- Code complet associé : notebook « 29 exemples algorithmes ».

Comparaison — régressions

- Linéaire : référence explicable
- Ridge / Lasso : régulariser, sélectionner
- Arbres / forêts : non-linéarités et interactions
- Boosting : meilleure précision possible, tuning plus délicat

Comparaison — classification

- Logistique : probabilités et interprétation
- KNN et SVM : importance de la mise à l'échelle
- Forêts : robustesse ; boosting : gain de performance
- Mesurer F1 et PR-AUC si les événements sont rares

Comparaison — non supervisé

- K-means : groupes compacts ; DBSCAN : formes arbitraires
- GMM : degrés d'appartenance ; CHA : hiérarchie
- ACP : variance globale ; t-SNE / UMAP : voisinages
- Isolation Forest : atypies non étiquetées

Comparaison — renforcement

- Bandits : actions sans état dynamique
- Q-learning : cible avec meilleur gain futur, hors politique
- SARSA : action réellement suivie, sur politique
- DQN : approximation neuronale, expérience et réseau cible

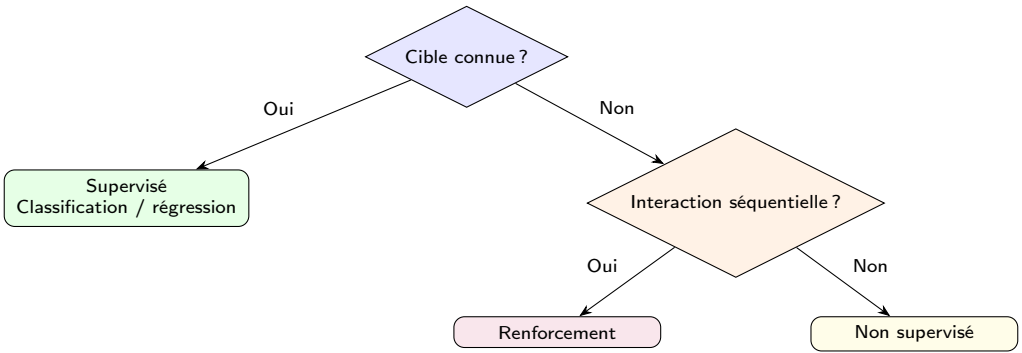
Arbre de décision pour choisir un modèle

- Cible numérique ? Régression ; cible catégorielle ? Classification
- Pas de cible ? Clustering, projection ou anomalie
- Actions et récompenses séquentielles ? Renforcement
- Données chronologiques : validation temporelle indispensable

Déploiement et contrôle

- Comparer au niveau de référence métier
- Conserver preprocessing et modèle ensemble
- Vérifier dérive des données, équité et calibration
- Documenter licences des jeux et sécurité des données

Schéma — guide de choix



TP 1 — Régression des dépenses

- Données publiques : Diabetes (scikit-learn)
- Approches : Dummy, Ridge, forêt aléatoire
- Prérequis : préparation, qualité, hypothèses et métriques
- Livrables : notebook, tableau comparatif, interprétation critique
- Deux versions séparées : étudiant et corrigé enseignant.

TP 2 — Risque de défaut de crédit

- Données publiques : UCI 350
- Approches : Dummy, logistique, forêt aléatoire
- Prérequis : préparation, qualité, hypothèses et métriques
- Livrables : notebook, tableau comparatif, interprétation critique
- Deux versions séparées : étudiant et corrigé enseignant.

TP 3 — Segmentation de données clients

- Données publiques : Wine (sans utiliser les étiquettes)
- Approches : K-means, CHA, ACP
- Prérequis : préparation, qualité, hypothèses et métriques
- Livrables : notebook, tableau comparatif, interprétation critique
- Deux versions séparées : étudiant et corrigé enseignant.

TP 4 — Q-learning sur FrozenLake

- Données publiques : Gymnasium FrozenLake-v1
- Approches : aléatoire, Q-learning, SARSA
- Prérequis : préparation, qualité, hypothèses et métriques
- Livrables : notebook, tableau comparatif, interprétation critique
- Deux versions séparées : étudiant et corrigé enseignant.

Exercice de révision 1

Question

Quelle différence entre paramètre et hyperparamètre ?

Corrigé dans le guide enseignant des TP.

Exercice de révision 2

Question

Pourquoi conserver un jeu de test final ?

Corrigé dans le guide enseignant des TP.

Exercice de révision 3

Question

Quand choisir PR-AUC plutôt que accuracy ?

Corrigé dans le guide enseignant des TP.

Exercice de révision 4

Question

Que révèle une forte erreur de test et faible erreur d'entraînement ?

Corrigé dans le guide enseignant des TP.

Exercice de révision 5

Question

Pourquoi standardiser avant SVM ou KNN ?

Corrigé dans le guide enseignant des TP.

Exercice de révision 6

Question

Que signifie le facteur gamma dans Q-learning ?

Corrigé dans le guide enseignant des TP.

Exercice de révision 7

Question

Pourquoi les clusters ne sont-ils pas des classes réelles ?

Corrigé dans le guide enseignant des TP.

Exercice de révision 8

Question

Que signifie une importante variance expliquée en ACP ?

Corrigé dans le guide enseignant des TP.

Exercice de révision 9

Question

Pourquoi Random Forest réduit-elle la variance des arbres ?

Corrigé dans le guide enseignant des TP.

Exercice de révision 10

Question

Quel risque dans une validation aléatoire de données temporelles ?

Corrigé dans le guide enseignant des TP.

Références de base

- Hastie, Tibshirani et Friedman — The Elements of Statistical Learning (2009).
- Géron — Hands-On Machine Learning (édition actualisée).
- Bishop — Pattern Recognition and Machine Learning.
- Murphy — Probabilistic Machine Learning.
- Sutton et Barto — Reinforcement Learning : An Introduction.
- scikit-learn.org ; gymnasium.farama.org ; archive.ics.uci.edu.

À retenir — synthèse

- Un algorithme s'évalue sur données réellement nouvelles.
- Une formule devient utile lorsque chaque symbole est compris.
- La simplicité est une référence, pas une faiblesse.
- Les métriques, l'éthique et les contraintes métier guident le choix final.